Lecture 11

Dynamic Memory Allocation

- We can have a situation in programming where the data is dynamic in nature. That is number of data item keep changing during the execution of the program.

  Example -> whatsApp chats messages

  Messages arrives and messages deleted , the count of message is not fixed.

- Such situation can be handled by using dynamic data structure in conjunction with dynamic memory management

continue…

- Dynamic data structures provide flexibility in adding, deleting or rearranging data items at run time. Example of dynamic data structures are linked list, stack, queue, tree and graph.
- Dynamic memory management techniques permit us to allocate additional memory space or to release unwanted space at run time, thus, optimizing the use of storage space.

continue…

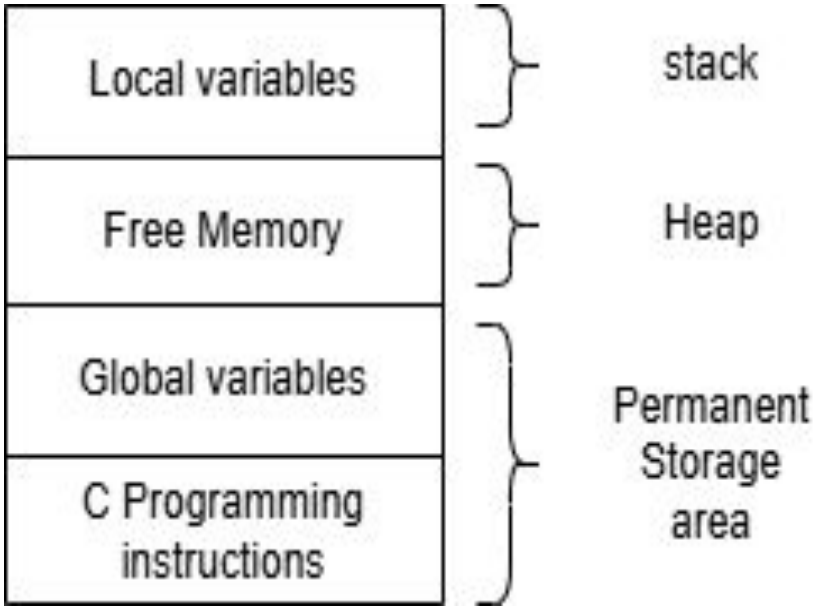
- The process of allocating memory at run time is known as dynamic memory allocation.
- Ther are four library routines known as memory management functions that can be used for allocating and freeing memory during program execution.

continue…

Function	Task
malloc	Allocates request size of bytes and returns a pointer to the first byte of allocated space.
calloc	Allocates space for an array of elements, initializes them to zero and then returns a pointer to the memory.
realloc	Modifies the size of previously allocated space.
free	Free previously allocated space

Memory Allocation Process

Storage of a C program



malloc

- The malloc function reserves a block of memory of specified size and returns a pointer of type void. This means that we can assign it to any type of pointer.

  Syntax: ptr = (cast-type *) malloc(byte-size);

- example1:

```
int *x;

x = (int *) malloc(100*sizeof(int));
```

- example2:

```
char *cptr;

Cptr = (char *) malloc(10);
```

continue…

- We may also use malloc to allocate space for complex data types such as structures. example:

  struct store *str_var;

  st_var = (struct store *)malloc(sizeof(struct store));

- malloc allocates a block of contiguous bytes. Allocation fails if space in heap is not sufficient to satisfy the request.
- If allocation of memory fails, it returns NULL.

calloc

- It is memory allocation function that is normally used for requesting memory space at run time for storing derived data types such as array and structures.
- malloc allocates a single block of storage space, calloc allocates multiple blocks of storage, each of same size and then sets all bytes to zero.
- syntax:

  ptr = (cast-type *)calloc(n, elem-size);

- It allocates contiguous space of n blocks, eack of size elem-size bytes.
- All bytes are initialized to zero and a pointer to the first byte of the allocated region is returned.
- If there is no enough space then a NULL pointer is returned.

continue…

Example ->

```
struct student{
        char name[20];
      float age;
      long int id_num;
};
typedef struct student record;
int class_size = 66;
record *ptr;
ptr = (record *)calloc(class_size, sizeof(record));
if(ptr == NULL){
      printf("not sufficient memory");
      exit(1);
}
```

free

- Compile time storage of a variable is allocated and released by the system in accordance with its storage class.
- With the dynamic run-time allocation, it is our responsibility to release the space when it is not required.

  free(ptr);

Dangling Pointer

- If you perform free the memory is deallocated but still ptr is pointing to that address. Meaning that it not the pointer being released but rather what it was pointing to.
- Pointer is pointing to a location which was released and now pointer is pointing to invalid location, which is problem.
- Initiallize pointer to NULL after deallocation of memory.

Example -> free(ptr);

ptr = NULL;

Memory Leak

- It happens when the pointer loses its initial allocated address due to which pointed memory block can not be accessed or can not be deallocated.

  Example ->

```
int main(){

    int *ptr = (int *)malloc(sizeof(int));

    *ptr = 10;

    printf(“%d”, *ptr);

    ptr = NULL;

    return 0;

    }
```

realloc

- It is likely that we discover later, the previously allocated memory is not sufficient and we need additional space for more elements. It is also possible that the memory allocated is much larger than necessary and we want to reduce it.
- In both cases, we can change the memory size already allocated with the help of realloc. This process is called memory reallocation.

  example-> ptr = malloc(size);

  ptr = realloc(ptr, newsize);//this is the reallocation of previous memory

continue…

- realloc allocates a new memory space of size newsize to the pointer variable ptr and returns pointer to the first byte of the new memory block.
- Note that the new memory block may or may not begin at the same place as the old one.
- If the function is unsuccessful in locating additional space, it returns a NULL pointer and the original block is freed(lost).

continue…

- example:

```
int main(){
        char *buffer;
if((buffer = (char *)malloc(10)) == NULL){
        printf("malloc failed.");
        exit(1);
 }
strcpy(buffer, "Hyderabad");
if((buffer = (char *)realloc(buffer, 15)) == NULL){
        printf("reallocation failed");
        exit(1);
        }
strcpy(buffer, "Secunderabad");
printf("%s", buffer);
free(buffer);
}
```

Self-Referential Structures

- Self-referential structures are those in which at least one member is a pointer to another structure of the same type as itself.
- Declaration of a self-referential structure is as follows:

```
struct example{

     declaration of other members;

     struct example *self_type_pointers;

}
```